Stat 184 Homework #4.3

Runcheng Lin

2026-04-12

Busiest Airports Analysis

For this section, I compared six of the world's busiest passenger airports from 2020 to 2025. The table below summarizes annual passenger traffic for ATL, FRA, PKX, LHR, DXB, and HND.

Table 1: Passenger Traffic for Six Major Airports, 2020–2025

Airport	2020	2021	2022	2023	2024	2025
ATL	42,918,945	75,211,943	93,700,000	104,650,000	108,067,766	106,302,208
FRA	18,768,601	24,500,000	48,500,000	59,700,000	70,600,000	56,185,219
PKX	16,090,000	25,300,000	47,000,000	52,000,000	62,000,000	70,000,000
LHR	22,574,669	19,400,000	61,700,000	79,100,000	83,882,140	84,463,000
DXB	25,900,000	29,000,000	66,100,000	86,700,000	92,331,506	95,200,000
HND	60,000,000	63,000,000	78,000,000	85,200,000	90,000,000	91,430,000

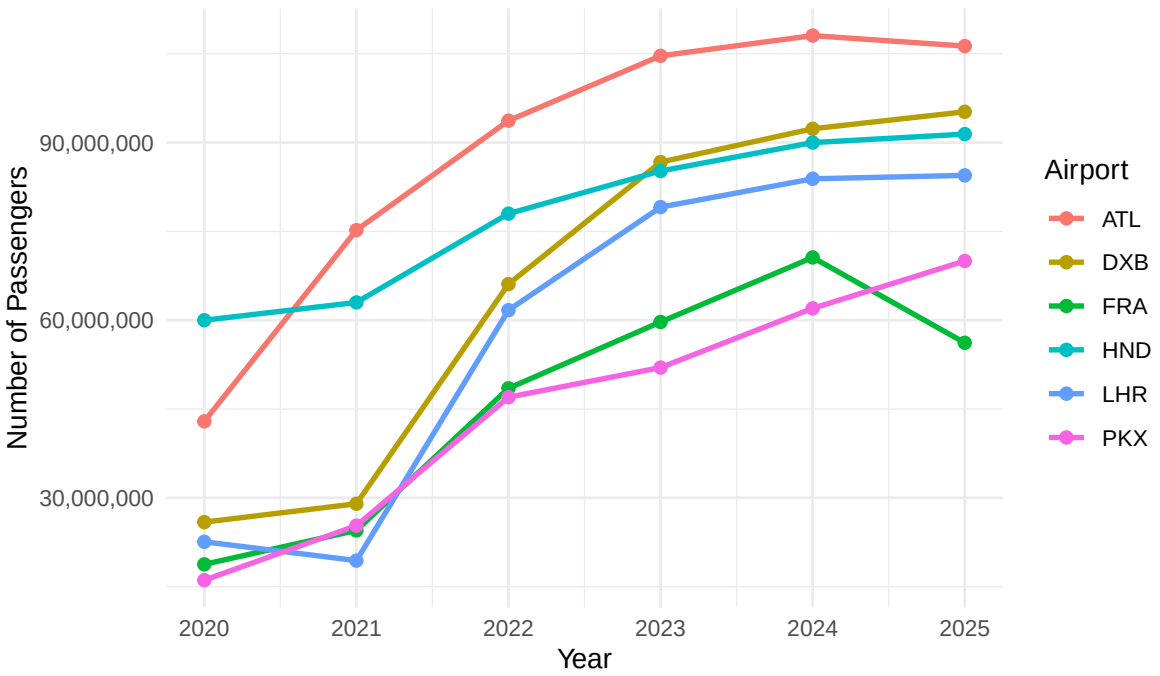


Figure 1: Passenger Traffic Trends for Six Major Airports, 2020–2025

The line plot shows that passenger traffic generally increased from 2020 to 2025 for all six airports, although the rate of change was different across locations. ATL remained the busiest airport in most years and showed a strong recovery after 2020. DXB and HND also had consistently high passenger totals and followed clear upward trends over the period.

LHR and PKX showed substantial growth as well, but their passenger volumes remained lower than ATL, DXB, and HND in most years. FRA increased through 2024 but then dropped in 2025, making its pattern less consistent than the others. Overall, the figure suggests that while all six airports experienced growth over time, the pace of recovery and later trends were not the same across airports. # Monte Carlo Numerical Integration In this section, I use Monte Carlo numerical integration to estimate the area under a curve. The Monte Carlo method works by generating random points inside a rectangular region and then calculating the proportion of points that fall below the function. That proportion is multiplied by the area of the rectangle to estimate the integral.

For this example, I used the function (f(x) = -(x-4)^2 + 4) on the interval (2 x 6). I then compared the simulation results at four different sample sizes: (n=10), (n=100), (n=1000), and (n=10000).

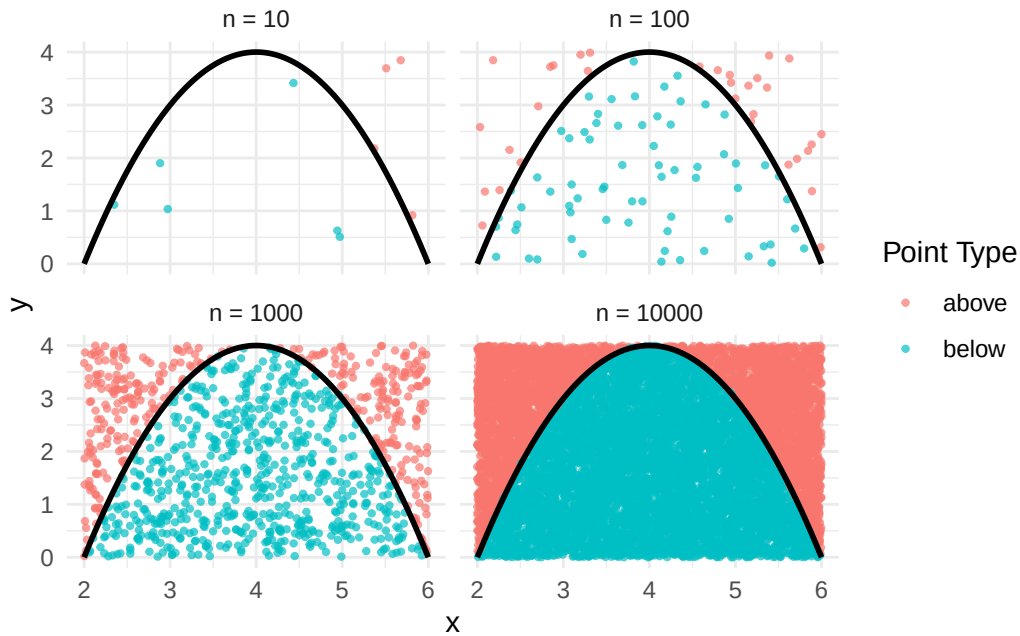


Figure 2: Monte Carlo Numerical Integration at Different Sample Sizes

Table 2: Monte Carlo Estimates at Different Sample Sizes

Sample Size	Proportion Below	Estimate	True Value	Absolute Error
n = 10	0.6000	9.6000	10.6667	1.0667
n = 100	0.6800	10.8800	10.6667	0.2133
n = 1000	0.6620	10.5920	10.6667	0.0747
n = 10000	0.6593	10.5488	10.6667	0.1179

The small multiples show that the Monte Carlo estimate becomes more stable as the number of random points increases. When the sample size is small, the estimate can vary substantially because the random points do not cover the rectangle very evenly. As the sample size increases, the point pattern becomes denser and the estimated area gets closer to the true value of the integral.

The results table also shows this pattern numerically. The estimate at (n=10) is less stable, while the estimates at (n=1000) and (n=10000) are generally much closer to the true value. This demonstrates that larger simulations tend to produce more reliable Monte Carlo estimates.

Planning and Prompting GenAI Tools

AI prompt with plan

Goal

Explore differences in ulnar calcium measurements across time and across the two treatment groups. ### Neads 1.keep the origin csv file 2.identify the treatment group and year 3.convert it to tidy data 4.Create a plot that can show the differences ### steps 1.read the origin csv 2.add an id variable for participants 3.rename the column; the first called year, the second called diet 4.reshape the data 5.create a plot that compares calcium measurements across time and treatment groups. 6.add labels and a caption so the graph is easy to interpret.

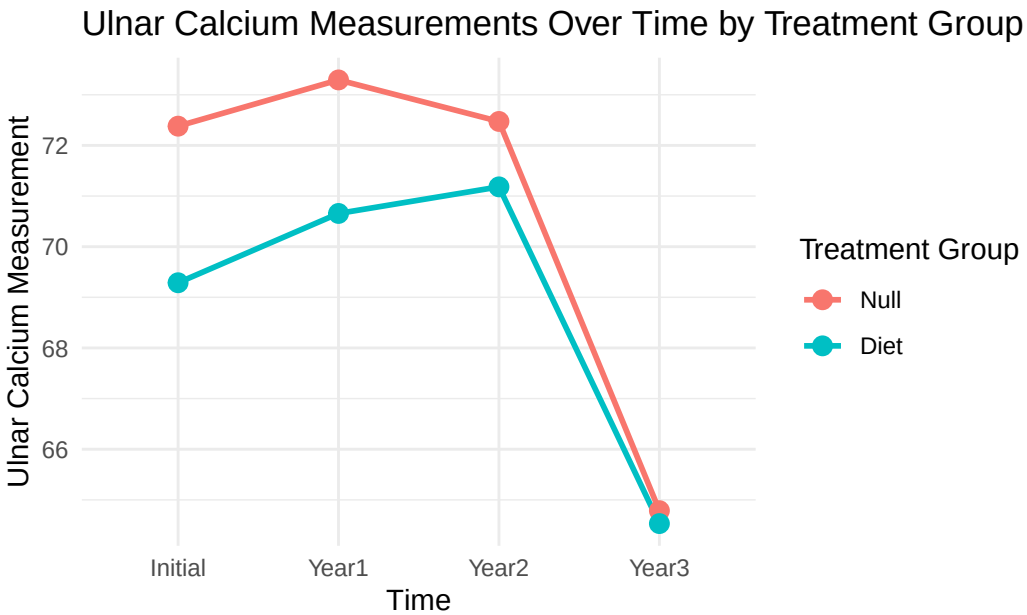
Text input

I am working with a CSV file called calcium.csv. The data come from a longitudinal study and include ulnar calcium measurements across four time points: Initial, Year 1, Year 2, and Year 3. The first four columns represent the null treatment group, and the second four columns represent the diet/exercise treatment group. The data are not tidy, and I should not modify the original CSV file before uploading it. My goal is to tidy the data and create a plot that helps compare ulnar calcium measurements across time and between the two treatment groups. Please follow the goal needs and steps: ## Goal Explore differences in ulnar calcium measurements across time and across the two treatment groups. ## Neads 1.keep the origin csv file 2.identify the treatment group and year 3.convert it to tidy data 4.Create a plot that can show the differences ## steps 1.read the origin csv 2.add an id variable for participants 3.rename the column; the first called year, the second called diet 4.reshape the data 5.create a plot that compares calcium measurements across time and treatment groups(By R). 6.add labels and a caption so the graph is easy to interpret. ### GPT answer Here is a clean way to do it in **R** without changing the original `calcium.csv` file.

You can use this workflow:

```
library(tidyverse)

# 1. Read the original csv file
calcium <- read_csv("calcium.csv")

# 2. Add an id variable for participants
calcium <- calcium %>%
  mutate(id = row_number())

# 3. Rename the columns:
```

```r
#    first 4 = Null group
#    second 4 = Diet/Exercise group
colnames(calcium)[1:8] <- c(
  "Initial_Null", "Year1_Null", "Year2_Null", "Year3_Null",
  "Initial_Diet", "Year1_Diet", "Year2_Diet", "Year3_Diet"
)

# 4. Reshape to tidy data
calcium_tidy <- calcium %>%
  pivot_longer(
    cols = -id,
    names_to = c("year", "diet"),
    names_sep = "_",
    values_to = "calcium"
  )

# Make year appear in the correct order in the plot
calcium_tidy <- calcium_tidy %>%
  mutate(
    year = factor(year, levels = c("Initial", "Year1", "Year2", "Year3")),
    diet = factor(diet, levels = c("Null", "Diet"))
  )

# 5. Create a plot comparing calcium measurements across time and treatment groups
ggplot(calcium_tidy, aes(x = year, y = calcium, color = diet, group = diet)) +
  stat_summary(fun = mean, geom = "line", linewidth = 1) +
  stat_summary(fun = mean, geom = "point", size = 3) +
  labs(
    title = "Ulnar Calcium Measurements Over Time by Treatment Group",
    x = "Time",
    y = "Ulnar Calcium Measurement",
    color = "Treatment Group",
    caption = "Points and lines show the mean ulnar calcium measurement at each time point fo
  ) +
  theme_minimal()
```



ulnar calcium measurement at each time point for each treatment group.

What this does:

- keeps the **original CSV file unchanged**
- adds a participant **id**
- identifies both **year** and **treatment group**
- converts the data into **tidy format**
- creates a plot that compares **calcium over time** between the **Null** and **Diet/Exercise** groups

The tidy dataset will look like this structure:

- `id` = participant number
- `year` = Initial, Year1, Year2, Year3
- `diet` = Null or Diet
- `calcium` = measurement value

If you want, I can also give you:

- a **boxplot version**
- a **spaghetti plot** for individual participants
- a **Quarto-ready chunk** for your homework submission

Conclusion

After implementing a reasonable plan and following the appropriate steps, the AI was able to complete the task. When only simple commands were given, the AI misunderstood the true intention. Therefore, it is important to make a reasonable plan.

Reflection

After completing this assignment, I realized that plans and steps are good helpers in achieving the goal. At the same time, I also realize that using captions can usually prevent display errors in documents.

GenAI Usage

This project used ChatGPT (OpenAI), GPT-5.4 Thinking as a generative AI tool. AI assistance was used only for:

helping draft narrative text, and helping with Part C of the assignment. Date: April 14 2026

Code

```r
library(tidyverse)
library(knitr)
library(scales)
```

```r
# Create dataset from wiki
airports <- tibble(
  airport = rep(c("ATL", "FRA", "PKX", "LHR", "DXB", "HND"), each = 6),
  year = rep(2020:2025, times = 6),
  passengers = c(
    # ATL
    42918945, 75211943, 93700000, 104650000, 108067766, 106302208,
    # FRA
    18768601, 24500000, 48500000, 59700000, 70600000, 56185219,
    # PKX
    16090000, 25300000, 47000000, 52000000, 62000000, 70000000,
    # LHR
    22574669, 19400000, 61700000, 79100000, 83882140, 84463000,
```

```r
    # DXB
    25900000, 29000000, 66100000, 86700000, 92331506, 95200000,
    # HND
    60000000, 63000000, 78000000, 85200000, 90000000, 91430000
  )
)
# Reshape the data
airport_table <- airports %>%
select(airport, year, passengers) %>%
pivot_wider(
names_from = year,
values_from = passengers
) %>%
mutate(across(`2020`:`2025`, comma))

# display
airport_table %>%
kable(
caption = "Passenger Traffic for Six Major Airports, 2020-2025",
col.names = c("Airport", "2020", "2021", "2022", "2023", "2024", "2025")
)
```

```r
#| fig-cap: "Passenger Traffic Trends for Six Major Airports, 2020-2025"
#| fig-width: 6.5
#| fig-height: 3.8
#| fig-pos: "H"
# generate the plot
ggplot(airports, aes(x = year, y = passengers, color = airport, group = airport)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    x = "Year",
    y = "Number of Passengers",
    color = "Airport"
  ) +
  theme_minimal()
```

```r
# set the function
f <- function(x) -(x - 4)^2 + 4
xmin <- 2
```

```r
xmax <- 6
ymin <- 0
ymax <- optimize(f, interval = c(xmin, xmax), maximum = TRUE)$objective
rect_area <- (xmax - xmin) * (ymax - ymin)

# dample sizes
n_values <- c(10, 100, 1000, 10000)

# generate Monte Carlo points
all_points <- map_dfr(n_values, function(n) {
  tibble(
    x = runif(n, xmin, xmax),
    y = runif(n, ymin, ymax)
  ) %>%
    mutate(
      fx = f(x),
      flag = if_else(y <= fx, "below", "above"),
      n = paste0("n = ", n)
    )
})

# create curve
curve_data <- tibble(
  x = seq(xmin, xmax, length.out = 400),
  y = f(x)
)

# Compute
mc_results <- all_points %>%
  group_by(n) %>%
  summarize(
    proportion_below = mean(flag == "below"),
    estimate = proportion_below * rect_area,
    .groups = "drop"
  ) %>%
  mutate(
    true_value = integrate(f, lower = xmin, upper = xmax)$value,
    absolute_error = abs(estimate - true_value)
  )
```

```r
#| fig-cap: "Monte Carlo Numerical Integration at Different Sample Sizes"
```

```r
ggplot(all_points, aes(x = x, y = y, color = flag)) +
  geom_point(size = 0.8, alpha = 0.7) +
  geom_line(
    data = curve_data,
    aes(x = x, y = y),
    inherit.aes = FALSE,
    linewidth = 1
  ) +
  facet_wrap(~ n, ncol = 2) +
  coord_cartesian(xlim = c(xmin, xmax), ylim = c(ymin, ymax)) +
  labs(
    x = "x",
    y = "y",
    color = "Point Type"
  ) +
  theme_minimal()
```

```r
mc_results %>%
  mutate(
    estimate = round(estimate, 4),
    true_value = round(true_value, 4),
    absolute_error = round(absolute_error, 4)
  ) %>%
  knitr::kable(
    caption = "Monte Carlo Estimates at Different Sample Sizes",
    col.names = c("Sample Size", "Proportion Below", "Estimate", "True Value", "Absolute Erro
  )
```